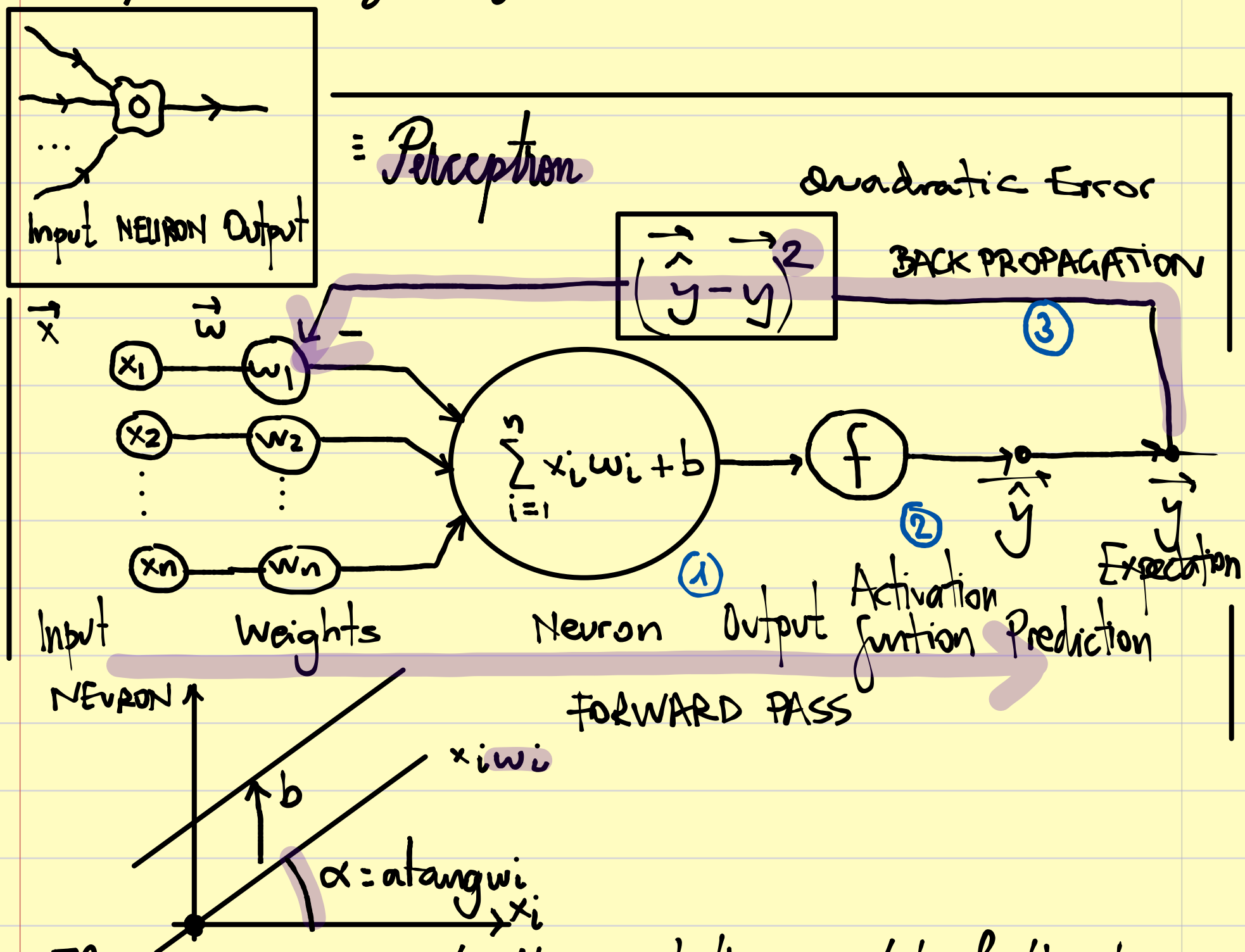


Deep Learning .. by hand



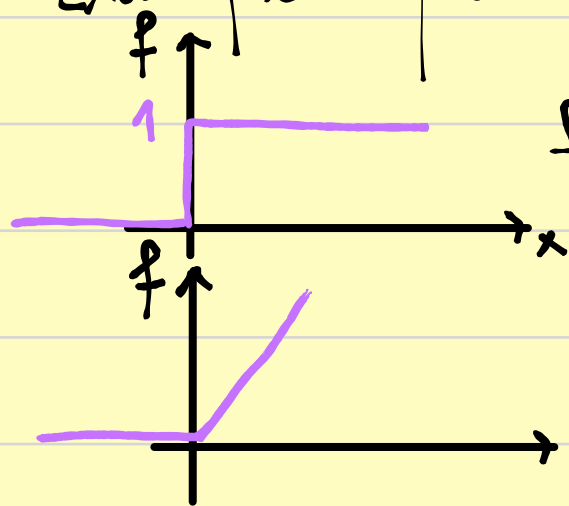
The neuron aggregates the input, the weights & the bias.

Step 1. ① Multiply all input values x_i with their weights w_i and calculate the weighted sum after the bias.

Step 2. ② Apply an activation function to the above, giving an output of the form

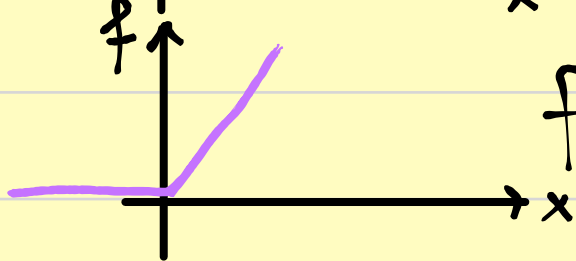
$$\hat{y} = f\left[\sum_{i=1}^n x_i w_i + b\right]$$

Examples of Activation Functions:



$$f(x) = \begin{cases} 0 & x \leq 0 \\ 1 & x > 0 \end{cases}$$

STEP FUNCTION



$$f(x) = \begin{cases} 0 & x \leq 0 \\ x & x > 0 \end{cases}$$

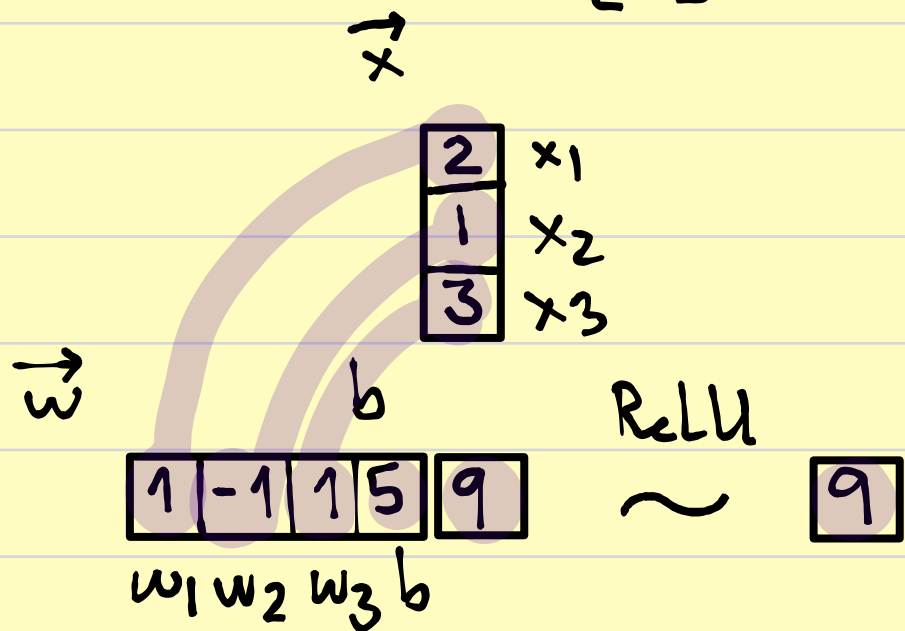
Rectifier linear Unit (ReLU)

Step 3. ③ We need to learn which weights make the equation $\hat{y} = f[\sum x_i w_i + b]$ as similar as possible to the expectation. For this we use the quadratic of the error (difference of the values). We define a cost function $C = \frac{1}{2} (y - \hat{y})^2$

$$C = \frac{1}{2} \left(y - f\left(\sum x_i w_i + b\right) \right)^2$$

Example 1. Single neuron FORWARD PASS.

$b=5$ $\vec{x} = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix}$ $\vec{w} = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}$ $f = \text{ReLU}$ output?



$$\sum_{i=1}^n w_i x_i + b =$$

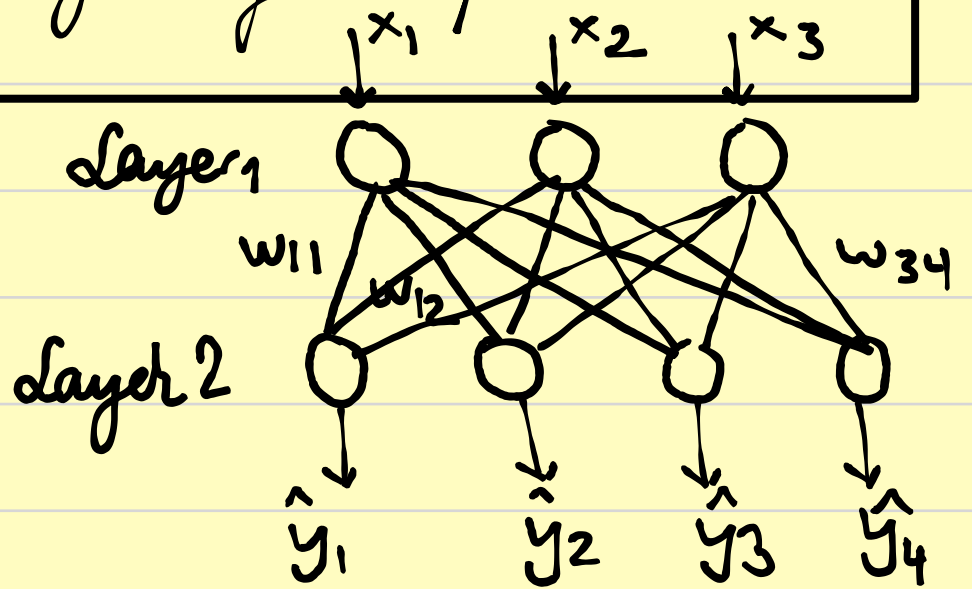
$$= 2 \cdot 1 + 1 \cdot (-1) + 3 \cdot 1 + 5 = 9$$

$$f\left(\sum x_i w_i + b\right) = f(9) = 9 = \hat{y}$$

ReLU

Example 2. Two layers of Perceptrons with 3 and 4 neurons.

$$\vec{x} = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix}$$



$$W_{3 \times 4} \quad b \quad \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix}$$

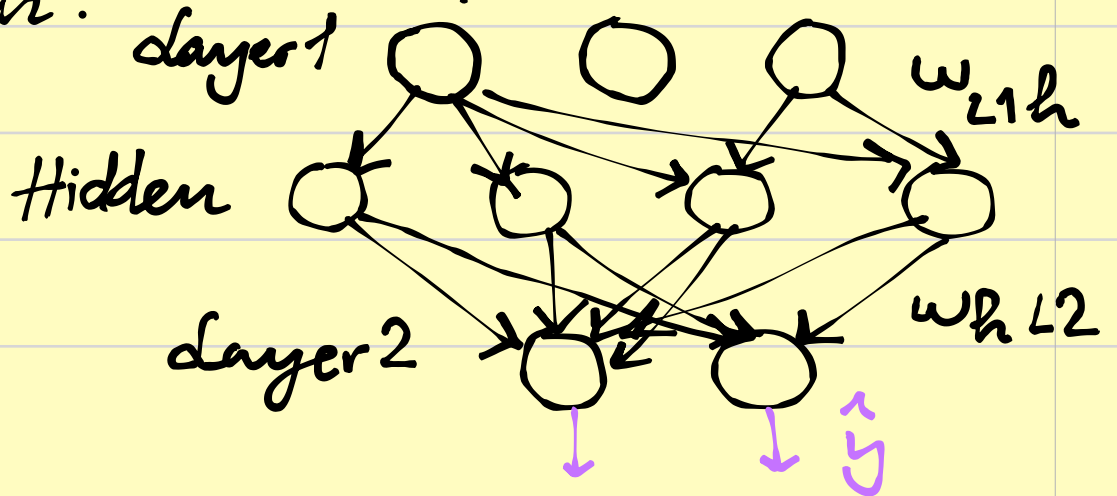
1	-1	1	5	9
1	1	0	0	3
0	1	1	1	5
1	0	1	-2	3

ReLU

$$\begin{bmatrix} 9 \\ 3 \\ 5 \\ 3 \end{bmatrix} \rightarrow \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \\ \hat{y}_4 \end{bmatrix}$$

- $2 \cdot 1 + 1 \cdot (-1) + 3 \cdot 1 + 5 = 9$
- $2 \cdot 1 + 1 \cdot 1 + 3 \cdot 0 + 0 = 3$
- $2 \cdot 0 + 1 \cdot 1 + 3 \cdot 1 + 1 = 5$
- $2 \cdot 1 + 1 \cdot 0 + 3 \cdot 1 + (-2) = 3$

Example 3. Three layers of perceptrons with $[3, 4, 2]$ neurons each.



L_1
 $w_{3 \times 4}$

b_1

3
1
2

1	-1	0	3	5
0	1	1	5	8
1	1	0	-2	2
-1	1	-1	-3	-9

ReLU

~

5
8
2
0

1	-1	1	0	3	2
-1	1	0	1	-2	1

ReLU

~

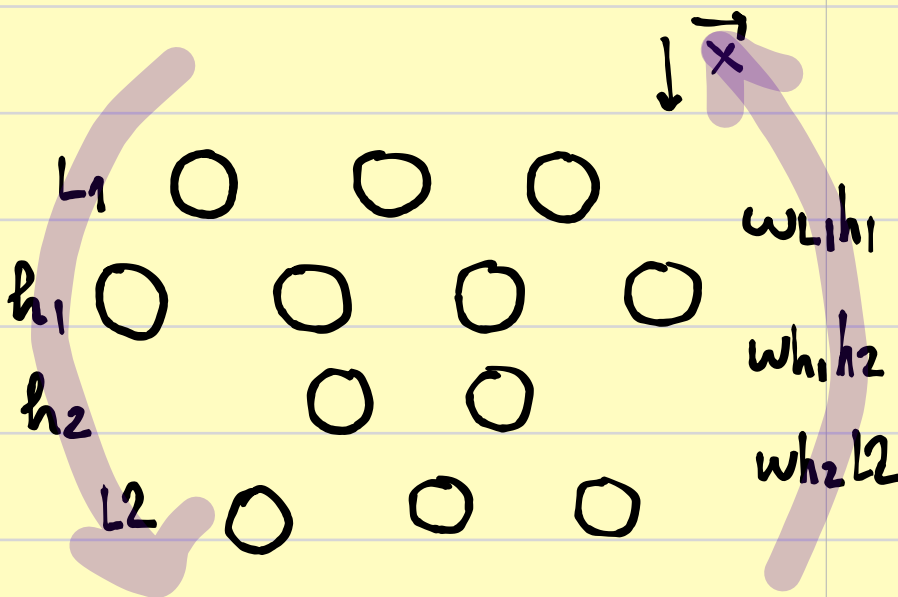
2
1

y

$w_{4 \times 2}$

b_2

Example 4. 4 Layers
 with $[3, 4, 2, 3]$ neurons with
 Forward pass and Back propagation.



2	1.88	0.12	-2.12	2
1	0.94	0.06	-1.06	1
3	2.82	0.18	3.18	3

$w_{L_1 h_1}^*$

b_1^*

$w_{L_1 h_1}$

b_1

1	1	-1	0	0
0	-1	1	1	3

ReLU

~

0
3

$w_{h_1 h_2}^*$

$w_{h_1 h_2}$

1	-1	1	3
2	-1	1	-3

7
3

7
3

-0.2	7
-0.8	3

-0.6	1
------	---

0.94	0.06	-1.06	1
------	------	-------	---

Error 1

b_2^*

$w_{h_1 h_2}$

1	-1	1	0	2
1	0	-1	1	-3

b_2

6
-7

ReLU

6
0

-3	0.36	2.64
0	0	0
-0.5	0.06	0.44

(3)

-0.6	1
------	---

Error 2

(4)

b_3^*

$w_{h_2 l_2}$

$$-0.5 \cdot 1 + 0.06 \cdot 0 + 0.44 \cdot 1 = -0.06$$

$$-0.5(-1) + 0.06 \cdot 1 + 0.44 \cdot 1 = 1$$

1	-1	2
0	1	-1
1	1	1

b_3

8
-1
7

(1)

0.5
0.06
0.44

SoftMax

1
0
0

y

-0.5
0.06
0.44

Error 3

(2)

(1) In the last layer before Back Propagation, we need the elements of the output vector to add up 1. [Probability Distribution]

$$\text{SoftMax} \begin{bmatrix} 8 \\ -1 \\ 7 \end{bmatrix} = \begin{bmatrix} \frac{8}{8+1+7} \\ \frac{1}{8+1+7} \\ \frac{7}{8+1+7} \end{bmatrix} = \begin{bmatrix} \frac{8}{16} \\ \frac{1}{16} \\ \frac{7}{16} \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0.0625 \\ 0.4375 \end{bmatrix}$$

- (2) The error becomes the bias of the previous layer.
- (3) Elementwise multiplication of the new bias & output.

(4) The error for the previous layer is obtained by multiplying the new bias with the old weights.

